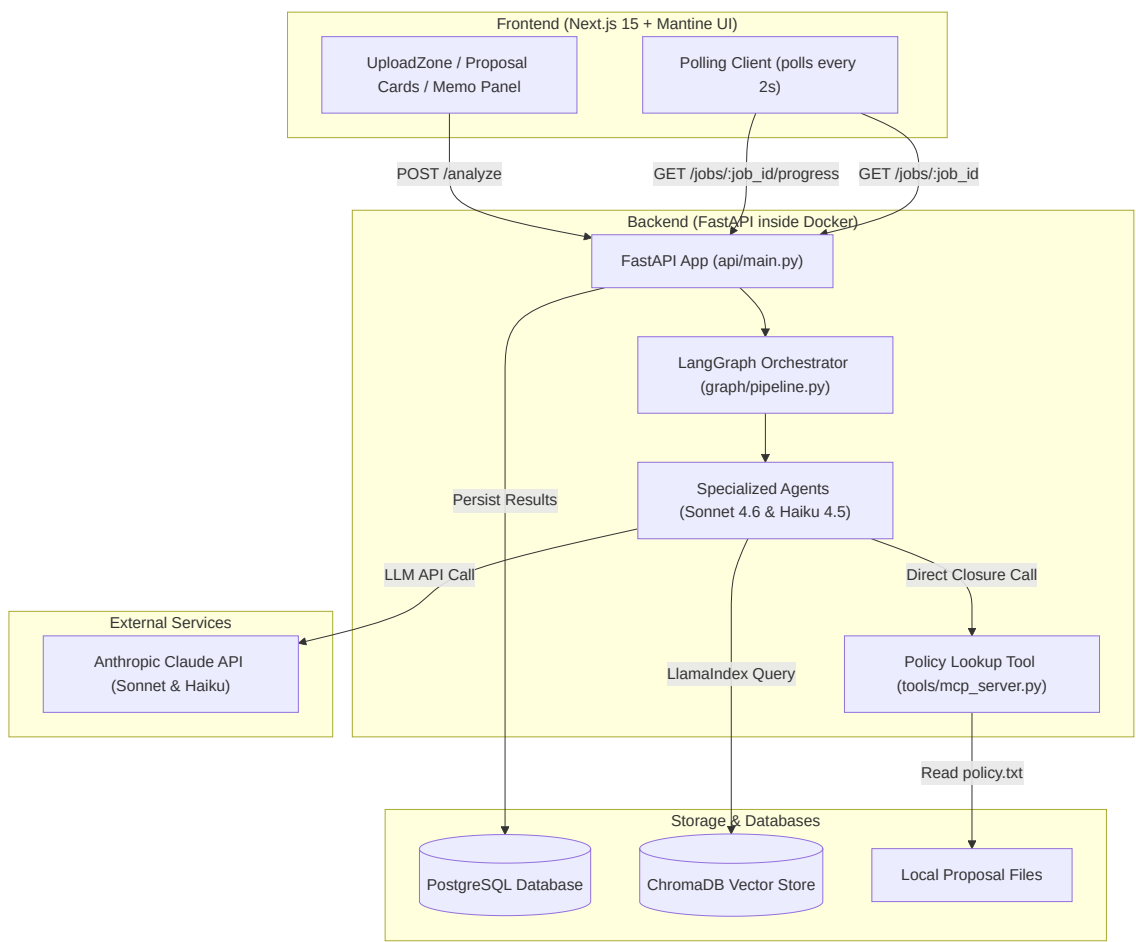


VendorLens Integration & System Architecture

This guide provides a comprehensive overview and visual systems diagrams detailing how the various components of VendorLens integrate. VendorLens is designed as a multi-agent contract evaluation system, built with a FastAPI backend, a Next.js frontend, a LangGraph orchestration layer, and local vector retrieval (RAG) capabilities.

1. High-Level Integration Architecture

The following diagram illustrates how the frontend, FastAPI backend, local vector store, relational database, and Anthropic LLM API interact during a proposal analysis lifecycle.



Key Components & Technical Choices

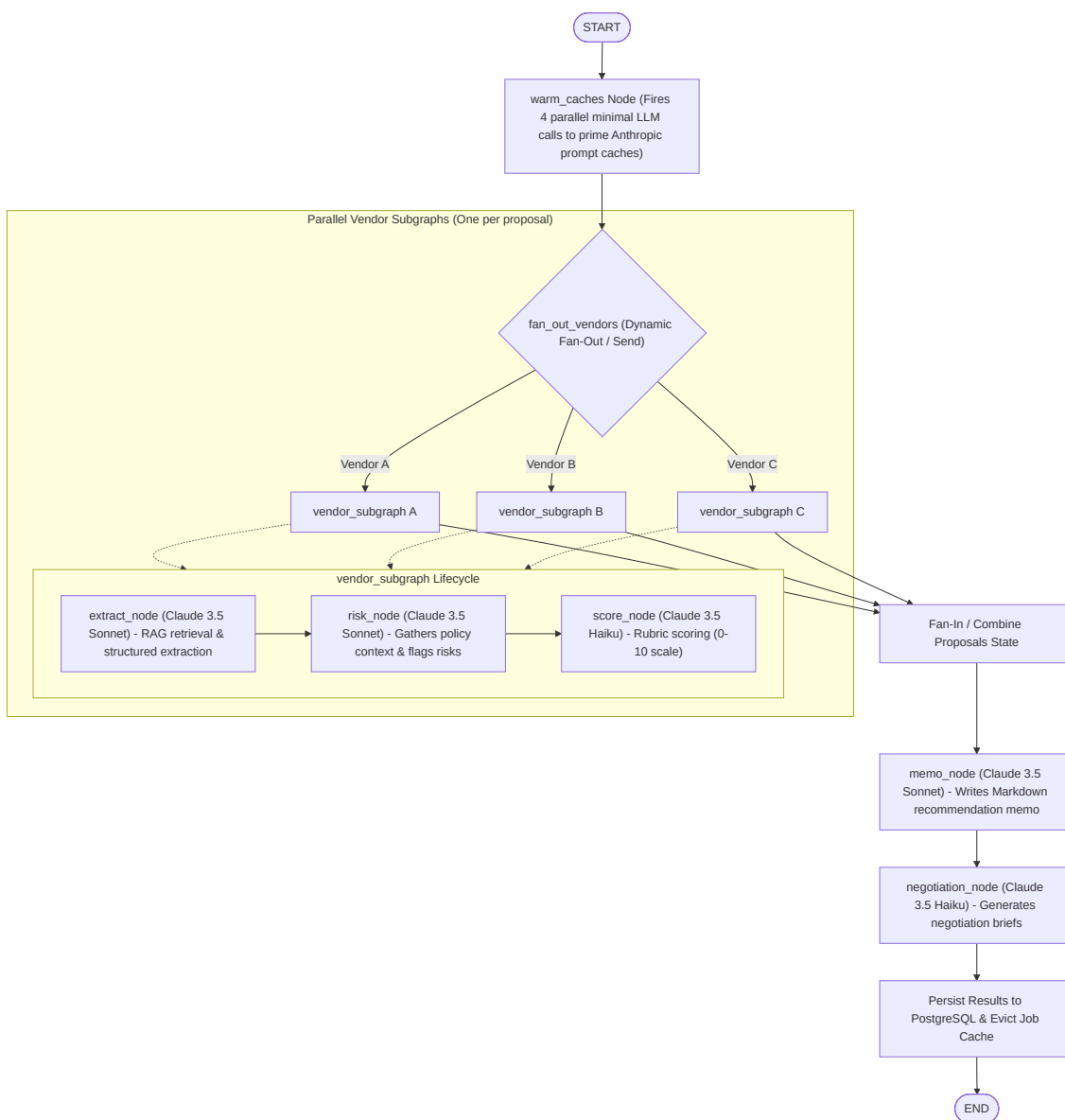
- **Next.js 15 + Mantine UI:** Rich, responsive frontend utilizing custom color palettes (maroon, blue, green, yellow scales) and interactive visual feedback (confetti, dynamic bars, tooltips). Polling occurs every 2 seconds to trace backend job progression.
- **FastAPI Backend (Dockerized):** Exposes endpoints for launching analyses (`/analyze`), checking progress (`/jobs/:job_id/progress`), and fetching completed reports (`/jobs/:job_id`).
- **PostgreSQL Database:** Contains the `analysis_runs` table, which maps a `job_id` to its status, current stage, and the compiled JSON result (proposals, risks, scores, memo, negotiation plans, and

actual token cost).

- **LlamaIndex + ChromaDB + FastEmbed:** Operates local embeddings (BAAI/bge-small-en-v1.5) via FastEmbed for the RAG pipeline. This requires no Google/Gemini API key and keeps embedding computation local.
- **Anthropic Claude API:** Utilizes Claude 3.5/3.6 Sonnet for complex reasoning and prose generation (Extraction, Risk, Memo agents), and Claude 3.5/4.5 Haiku for fast, structured numeric tasks (Scoring, Negotiation agents).

2. LangGraph Pipeline Topology & Agent Workflow

The LangGraph engine coordinates execution states and handoffs. The parent graph dynamically spins up parallel subgraphs for each vendor, gathers their extracted states, and routes them to consecutive compilation nodes.



Pipeline Node Roles & Models

Node	Model	Input	Output / State Modification
warm_caches	Sonnet / Haiku	PipelineState	Pre-warms system prompt caches concurrently (max_workers=4). Exits early for single-vendor runs to prevent wasteful calls.
extract_node	Sonnet	filename	Extracts contract structures (ProposalData) via LlamaIndex queries.
risk_node	Sonnet	ProposalData	Identifies security/compliance mismatches (RiskFlag array) using context-lookup.
score_node	Haiku	ProposalData + RiskFlag[]	Scores the proposal across 9 rubric dimensions (ScoreCard, 0-10 scale).
memo_node	Sonnet	ProposalState[]	Generates a Markdown recommendation memo based on scoring leaders and high-risk flags.
negotiation_node	Haiku	ProposalState[]	Generates BATNA guidelines, concessions, and vendor tactics (NegotiationBrief[]).

3. Optimizations & Technical Implementations

A. Cost Tracking Across Concurrent Sub-threads

- **The Problem:** A thread-local variable (`threading.local`) is isolated to the parent thread. When `LangGraph` fans out to parallel sub-threads (executing `extract_node` , `risk_node` , and `score_node` in a `ThreadPoolExecutor`), the cost tracking run ID is lost, resulting in zero reported costs.
- **The Solution:** We utilize `contextvars.ContextVar` to represent the current run ID. Because Python 3.11's default executor does not propagate context variables automatically to spawned threads, each pipeline node explicitly calls `_current_run_id.set(run_id)` at start-up.
- **Outcome:** Every API response's token usage (input, output, cached, read) is calculated and accumulated accurately across concurrent threads. The final USD cost is persisted in the PostgreSQL run record and rendered on the UI footer.

B. Pre-warming & Cache Bypass

- **Pre-warming:** For multi-vendor runs, the `warm_caches` node starts a `ThreadPoolExecutor` with `max_workers=4` to execute concurrent minimal calls, priming Anthropic caches for all four agents (Extraction, Risk, Scoring, Negotiation).
- **Single-Vendor Skip:** If only one proposal is submitted, parallel fan-out is unnecessary. The `warm_caches` node automatically exits early (`len(pending) < 2`), skipping cache warm-up. This avoids 4 redundant API calls, saving latency and tokens when caching offers no benefits.

C. Negotiation Token Optimization

- The `NegotiationAgent` receives a batched call containing all vendor profiles. Sending the full extracted `ProposalData` model (including granular features like `social_listening` , `mobile_app` , and `training_offered`) wastes prompt tokens.

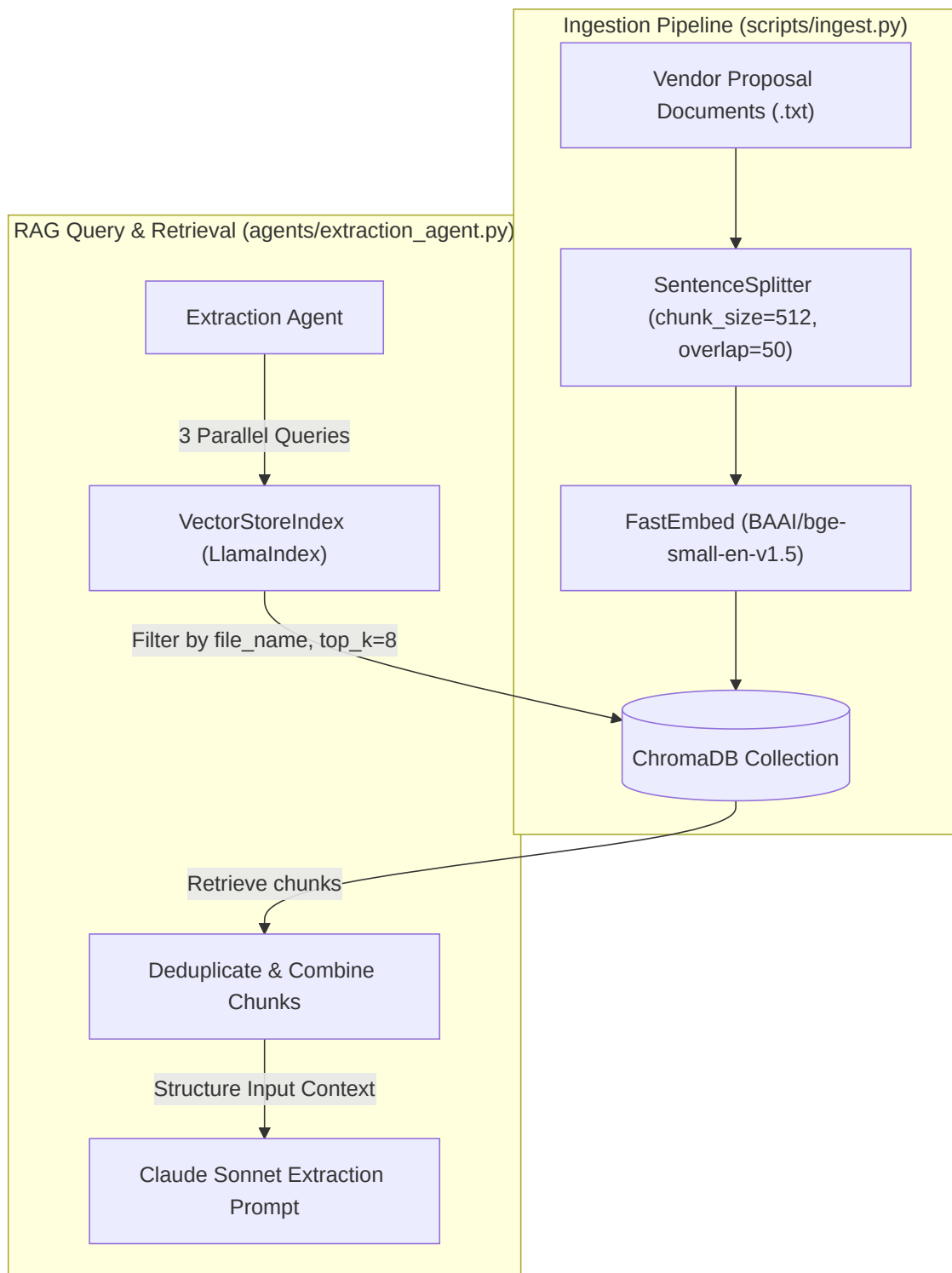
- The agent filters the input down to **16 critical commercial/legal fields** (such as `total_cost` , `termination_clause` , `liability_cap` , and `governing_law`). This reduces the batched token payload to the Negotiation Agent by **~45%**.

D. Scorecard Hover Tooltips

- To bridge the gap between technical metrics and business logic, the Next.js frontend renders dotted underlines on scorecard dimensions in both the `ProposalCard` and `VendorComparisonTable` .
 - Hovering over any metric displays a Mantine `Tooltip` outlining the exact criteria being evaluated (e.g. explaining what compliance standards, integrations, or security frameworks are under review).
-

4. RAG Pipeline & Context Retrieval Flow

The Retrieval-Augmented Generation (RAG) system loads vendor documentation, chunks it, and creates vector stores local to the system. During analysis, retrieval queries are deduplicated and piped to the Extraction Agent's context.



5. Swappable Context Bundles

VendorLens maintains decoupling between execution logic and domain knowledge. Swapping the procurement profile is done entirely via filesystem assets under `data/context_bundles/`.

```
data/context_bundles/
├─ lms/
│   ├─ policy.txt          <-- Procurement & liability rules
│   ├─ rfp_criteria_lms.txt <-- Scoring weights and metrics
│   └─ scoring_rubric_lms.txt <-- Scoring rubrics (0-10 scale)
├─ payroll/
│   ├─ policy.txt
│   ├─ rfp_criteria_payroll.txt
│   └─ scoring_rubric_payroll.txt
└─ erp/
    ├─ policy.txt
    ├─ rfp_criteria_erp.txt
    └─ scoring_rubric_erp.txt
```

[!NOTE] Swapping or adding a bundle does not require rewriting python agent code or re-compiling graph topologies. Registry mapping is defined in `tools/context_loader.py` .